



Universidad Experimental Nacional de Guayana

Vicerrectorado Académico

Coordinación General de Pregrado

P.F.G: Ingeniería en Informática

Unidad Curricular: Lenguaje y Compiladores

Tema 3 (Lenguajes y Gramáticas Formales)

Profesor:

Marquez Felix

Estudiantes:

Cortez Samuel V- 27.158.842

Duarte Alejandro V-28.530.657

Moreno Alondra V-30.331.870

Perdomo Arnaldo V-30.791.551

Ciudad Guayana, junio 2026

Índice

Introducción.....	3
2.1.- Fundamentos y Jerarquía (Teoría Aplicada).....	5
2.2.1.- Relación Gramática-Lenguaje: Concepto y Mecanismo de Generación.....	5
Concepto de la Relación.....	5
El Proceso de Derivación Paso a Paso.....	6
2.2.2 Definición de la Jerarquía de Chomsky.....	7
Los Cuatro Tipos de Gramáticas.....	8
Tipo 0 — Gramáticas Sin Restricción.....	8
Tipo 1 — Gramáticas Sensibles al Contexto.....	9
Tipo 2 — Gramáticas Libres de Contexto.....	10
Tipo 3 — Gramáticas Regulares.....	11
Tabla Comparativa de los 4 Tipos.....	13
Análisis Léxico — Tipo 3.....	14
Análisis Sintáctico — Tipo 2.....	14
Análisis Semántico — Tipos 0 y 1 en teoría.....	15
Código Fuente.....	16
Capturas de Pantalla del Simulador.....	18
2.2.1 Gramáticas y Geometría Computacional.....	20
Geometría de la Tortuga (Turtle Geometry).....	21
Análisis Semántico y Sintáctico.....	21
Fundamentos Matemáticos: El Cierre de Polígonos.....	21
2.3.- Higiene y Optimización de Gramáticas.....	22
2.3.1.- Patologías de las Gramáticas.....	22
a) Una Gramática Ambigua.....	23
b) Recursividad por la Izquierda.....	24
c) Factorización por la Izquierda.....	26
2.4 PGN Ajedrez + Regex + AFD.....	27
2.4.2. Definición de un subconjunto simplificado del lenguaje PGN.....	27
2.4.3. Diseño de la Expresión Regular (Regex).....	28
Para el subconjunto definido anteriormente, el diseño se fundamenta en la capacidad de las expresiones regulares para identificar tokens específicos.....	28
A. Expresión Regular (Regex).....	29
B. Autómata Finito Determinístico (AFD).....	29
Diagrama del AFD (Representación Lógica).....	29
Conclusión.....	32
Referencias.....	34

Introducción

Los lenguajes formales y las gramáticas constituyen uno de los pilares fundamentales sobre los que se asienta la ingeniería del software y, en particular, el diseño de compiladores e intérpretes. La capacidad de describir de manera precisa y rigurosa la sintaxis de un lenguaje de programación, así como de reconocer y generar cadenas de caracteres conforme a reglas bien definidas, es esencial para la construcción de herramientas que procesan, analizan y transforman el código fuente. En este contexto, la jerarquía de Chomsky, propuesta por Noam Chomsky en la década de 1950, ofrece un marco teórico que clasifica las gramáticas en cuatro niveles desde las sin restricción hasta las regulares y establece una correspondencia directa con modelos computacionales de creciente complejidad, como las máquinas de Turing, los autómatas linealmente acotados, los autómatas de pila y los autómatas finitos. Esta clasificación no solo tiene un valor conceptual, sino que encuentra una aplicación práctica inmediata en las fases de un compilador: el análisis léxico se apoya en gramáticas regulares (Tipo 3), el análisis sintáctico en gramáticas libres de contexto (Tipo 2), y las verificaciones semánticas más complejas se asocian, en teoría, con gramáticas sensibles al contexto (Tipo 1) o sin restricción (Tipo 0).

El presente informe aborda, desde una perspectiva teórica y aplicada, los fundamentos de los lenguajes y gramáticas formales, explorando su relación con la generación de cadenas, la definición de la jerarquía de Chomsky y su uso en la construcción de compiladores. Además, se extiende el estudio a dominios especializados, como la geometría computacional mediante lenguajes de dominio específico (DSL) para el control de una “tortuga gráfica”, y la notación estándar de partidas de ajedrez (PGN), donde se aplican expresiones regulares y autómatas finitos determinísticos para validar movimientos. Asimismo, se analizan las

patologías gramaticales más comunes ambigüedad, recursividad por la izquierda y factorización por la izquierda y se proponen técnicas de optimización para garantizar un análisis sintáctico determinístico y eficiente. Como complemento práctico, se ha desarrollado un simulador visual interactivo en Python que permite experimentar con los cuatro tipos de gramáticas, validando cadenas de entrada y generando ejemplos, lo que facilita la comprensión de los conceptos abstractos a través de la interacción directa.

El trabajo combina, por tanto, la exposición rigurosa de los conceptos teóricos con su materialización en herramientas y ejemplos concretos, reflejando la importancia de los lenguajes formales en la informática moderna y su impacto en el desarrollo de sistemas de procesamiento de lenguajes.

2.1.- Fundamentos y Jerarquía (Teoría Aplicada)

2.2.1.- Relación Gramática-Lenguaje: Concepto y Mecanismo de Generación

Concepto de la Relación

En las ciencias de la computación, un lenguaje formal (L) no se concibe como un idioma humano convencional, sino como un conjunto matemático, potencialmente infinito, de cadenas de caracteres finitas (palabras) construidas a partir de un alfabeto bien definido. Por otro lado, una gramática formal (G) es el motor generativo o el sistema de reglas finitas que define la estructura sintáctica exacta de ese lenguaje.

La relación entre ambas es de definición y pertenencia: la gramática actúa como el molde sintáctico o el esqueleto que delimita matemáticamente si una secuencia de texto arbitraria (código fuente) pertenece o no al lenguaje legítimo.

Ejemplo Práctico: > En el desarrollo de software, la relación se evidencia de forma cotidiana en la estructura de los datos. Si se define un lenguaje formal L como *"el conjunto de todos los objetos JSON válidos"*, la gramática formal asociada será el conjunto de reglas que dictan que un objeto debe abrir con {, cerrar con }, y contener pares de clave: valor separados por comas. Un compilador o parser utiliza esta gramática para validar el archivo; si la cadena respeta las reglas gramaticales, se acepta como parte del lenguaje.

Mecanismo de Generación de un Lenguaje

Para comprender cómo una gramática genera un lenguaje, se debe descomponer la gramática en su definición matemática formal. Una gramática G se define como una cuádrupla:

$$G = (V_N, V_T, P, S)$$

Donde:

- V_N Es el conjunto finito de símbolos no terminales (variables intermedias que se deben sustituir).
- V_T es el conjunto finito de símbolos terminales (los caracteres reales que forman el alfabeto del lenguaje, Σ).
- P es el conjunto finito de reglas de producción (fórmulas de sustitución del tipo $\alpha \rightarrow \beta$).
- S es el símbolo inicial ($S \in V_N$), el punto de partida de cualquier derivación.

El Proceso de Derivación Paso a Paso

El mecanismo mediante el cual la gramática da vida al lenguaje se denomina derivación y opera bajo el siguiente flujo lógico:

1. **Punto de Partida (S):** El proceso inicia obligatoriamente con el símbolo axiomático inicial S .
2. **Aplicación de Producciones ($\rightarrow$):** Se selecciona un símbolo no terminal presente en la cadena actual y se busca una regla en P que dicte cómo sustituirlo.
3. **Evaluación de Cadenas Intermedias:** El proceso se repite sucesivamente. Mientras la cadena contenga al menos un símbolo no terminal (V_N), se considera una forma de frase intermedia y no una palabra del lenguaje.
4. **Condición de Parada (Clausura):** Una derivación finaliza con éxito únicamente cuando la cadena resultante está compuesta exclusivamente por símbolos terminales (V_T). En ese instante, la cadena se ha convertido en una "palabra" válida.

Matemáticamente, el lenguaje generado por la gramática G , denotado como $L(G)$, es el conjunto de todas las cadenas de terminales que pueden derivarse desde el símbolo inicial S tras aplicar un número finito de pasos de producción (*):

$$L(G) = \{ \omega \in V_T^* \mid S \xrightarrow{*} \omega \}$$

Si una cadena de texto no puede ser alcanzada mediante ninguna combinación posible de las reglas de producción partiendo desde S , dicha cadena se declara sintácticamente inválida y queda excluida del lenguaje.

2.2.2 Definición de la Jerarquía de Chomsky

La Jerarquía de Chomsky es una clasificación formal de las gramáticas y los lenguajes formales, propuesta por el lingüista y filósofo Noam Chomsky en la década de 1950 (Chomsky, 1956). Su contribución central fue demostrar que no todas las gramáticas tienen el mismo poder generativo, es decir, que existe una relación directa entre la complejidad de las reglas de producción de una gramática y la potencia computacional del autómata necesario para reconocer el lenguaje que esa gramática describe.

La jerarquía organiza las gramáticas en cuatro tipos, denominados del Tipo 0 al Tipo 3, estableciendo una relación de inclusión estricta entre ellos. Formalmente, esta relación se expresa como: Tipo 3 $\subset$ Tipo 2 $\subset$ Tipo 1 $\subset$ Tipo 0. Esto significa que todo lenguaje regular (Tipo 3) es también libre de contexto (Tipo 2), y así sucesivamente hacia arriba. Sin embargo, la inversa no se cumple: existen lenguajes Tipo 2 que no pueden ser descritos por ninguna gramática Tipo 3.

Los Cuatro Tipos de Gramáticas

Tipo 0 — Gramáticas Sin Restricción

Definición Formal

Las gramáticas Tipo 0, también denominadas gramáticas sin restricción o gramáticas irrestrictas, son el nivel más general de la jerarquía. En estas gramáticas, las producciones tienen la forma $\alpha \rightarrow \beta$, donde tanto α como β son cadenas arbitrarias de terminales y no-terminales, sujetas únicamente a la condición de que α contenga al menos un símbolo no-terminal (Hopcroft et al., 2007). No existe ninguna restricción adicional sobre la longitud o la estructura de las producciones.

Autómata Equivalente

El autómata equivalente a las gramáticas Tipo 0 es la Máquina de Turing, propuesta por Alan Turing en 1936. Esta máquina representa el modelo computacional más poderoso conocido, capaz de simular cualquier algoritmo computable. Dispone de una cinta de memoria teóricamente infinita y un cabezal de lectura/escritura que puede moverse en ambas direcciones.

Capacidad y Limitaciones

finalmente sin producir una respuesta.

Ejemplo en Notación BNF

El siguiente ejemplo ilustra una gramática Tipo 0. La producción $\langle A \rangle \langle B \rangle ::= \langle B \rangle \langle A \rangle$ tiene múltiples no-terminales en el lado izquierdo, lo cual no está permitido en tipos inferiores de la jerarquía: Las gramáticas Tipo 0 reconocen exactamente la clase de los Lenguajes Recursivamente Enumerables (LRE). Una limitación práctica fundamental es que estas

gramáticas no garantizan la terminación del proceso de reconocimiento: el autómata puede ejecutarse inde

```
<S>      ::= <A><B> | ε
<A><B> ::= <B><A>
<A>      ::= a
<B>      ::= b
```

Tipo 1 — Gramáticas Sensibles al Contexto

Definición Formal

Las gramáticas Tipo 1, o Gramáticas Sensibles al Contexto (GSC), introducen la primera restricción significativa sobre las producciones. Estas tienen la forma $\alpha A \beta \rightarrow \alpha \gamma \beta$, donde A es un único símbolo no-terminal, α y β son cadenas de contexto que pueden ser vacías, y γ es una cadena no vacía. La restricción fundamental es que la longitud del lado derecho nunca puede ser menor que la del izquierdo: $|\alpha| \leq |\beta|$ para toda producción (Sipser, 2013). Esto implica que nunca se puede reducir la longitud de la cadena durante una derivación.

Autómata Equivalente

El autómata equivalente es el Autómata Linealmente Acotado (ALA), que puede concebirse como una Máquina de Turing con la restricción de que solo puede utilizar una porción de la cinta proporcional al tamaño de la entrada. Esta limitación de memoria lo distingue del poder irrestricto del Tipo 0.

Caso Práctico — Concordancia Gramatical

Un ejemplo clásico de lenguaje sensible al contexto es $\{a^n b^n c^n \mid n \geq 1\}$: cadenas formadas por n ocurrencias de 'a', seguidas de n 'b' y n 'c'. Este lenguaje no puede ser reconocido por una gramática Tipo 2 porque requiere verificar simultáneamente tres grupos de símbolos con la misma longitud. En lenguaje natural, la concordancia gramatical presenta propiedades similares: 'el gato come' es válido, mientras que 'los gato come' viola la concordancia de número.

Ejemplo en Notación BNF

```
S → aSBC | aBC
CB → BC
aB → ab
bB → bb
bC → bc
cC → cc
```

Tipo 2 — Gramáticas Libres de Contexto

Definición Formal

Las Gramáticas Libres de Contexto (GLC), o gramáticas Tipo 2, son el nivel más relevante en la práctica de ingeniería de compiladores. Toda producción tiene la forma $A \rightarrow \alpha$, donde A es un único símbolo no-terminal y α es cualquier cadena de terminales y no-terminales, incluyendo la cadena vacía. La denominación 'libre de contexto' indica que la regla de sustitución para A se aplica independientemente del contexto en que aparezca A en la cadena (Aho et al., 2007).

Autómata Equivalente

El autómata equivalente es el Autómata de Pila (Pushdown Automaton, PDA). A diferencia del autómata finito, el PDA dispone de una memoria adicional en forma de pila (estructura LIFO), lo que le permite manejar estructuras anidadas y recursivas como paréntesis balanceados o bloques de código. Esta memoria de pila es precisamente lo que hace posible el análisis de la mayoría de los lenguajes de programación modernos.

Uso en Compiladores

Las GLC son la base del Análisis Sintáctico (Parsing), la fase del compilador que verifica que la secuencia de tokens producida por el analizador léxico siga la gramática del lenguaje. El resultado de esta fase es el Árbol de Sintaxis Abstracta (AST). Herramientas ampliamente utilizadas como ANTLR, Yacc y Bison implementan parsers basados en GLC.

Ejemplo en Notación BNF — Expresiones Aritméticas

```
<expr> ::= <expr> '+' <term>
        | <expr> '-' <term>
        | <term>

<term>  ::= <term> '*' <factor>
        | <term> '/' <factor>
        | <factor>

<factor> ::= '(' <expr> ')'
        | número
        | identificador
```

Tipo 3 — Gramáticas Regulares

Definición Formal

Las gramáticas Tipo 3, o Gramáticas Regulares, representan el nivel más restringido de la jerarquía. Solo se permiten producciones de la forma $A \rightarrow aB$ (lineal por la derecha) o $A \rightarrow a$, donde A y B son símbolos no-terminales y a es un terminal. También se admite la variante lineal por la izquierda: $A \rightarrow Ba$. Es importante señalar que ambas formas no pueden mezclarse dentro de la misma gramática (Hopcroft et al., 2007).

Autómata Equivalente

El autómata equivalente es el Autómata Finito, ya sea en su versión determinística (AFD) o no determinística (AFND). Este autómata opera con memoria constante: su comportamiento en cada paso depende únicamente del estado actual y del símbolo leído, sin ninguna memoria auxiliar. Su principal ventaja es la eficiencia: opera en tiempo lineal $O(n)$ con respecto al tamaño de la entrada y con memoria constante $O(1)$.

Uso en Compiladores

Las gramáticas regulares son la base del Análisis Léxico, la primera fase de un compilador. En esta fase, el código fuente se convierte en una secuencia de tokens (unidades léxicas): palabras clave, identificadores, números, operadores y delimitadores. Herramientas como Flex y Lex implementan lexers basados en expresiones regulares, que son equivalentes a las gramáticas Tipo 3.

Ejemplo en Notación BNF — Identificadores

```

<id> ::= <letra>
      | <id> <letra>
      | <id> <dígito>

<letra> ::= 'a' | 'b' | ... | 'z'
          | 'A' | 'B' | ... | 'Z'
          | '_'

<dígito> ::= '0' | '1' | ... | '9'

Expresión Regular equivalente: [a-zA-Z_][a-zA-Z0-9_]*

```

AFD para Identificadores — 3 Estados

El siguiente diagrama describe el Autómata Finito Determinístico equivalente: el estado q_0 es el estado inicial; al leer una letra o '_', se transiciona a q_1 (estado de aceptación); desde q_1 , al leer letra, dígito o '_', se permanece en q_1 ; al leer cualquier otro símbolo desde q_0 o q_1 , se transiciona a q_E (estado de error).

Tabla Comparativa de los 4 Tipos

La siguiente tabla resume las características fundamentales de cada nivel de la Jerarquía de Chomsky:

Tipo	Nombre	Restricción	Autómata	Ejemplo
0	Sin Restricción (Irrestricta)	$\alpha \rightarrow \beta$ (sin límites)	Máquina de Turing	Lenguajes recursivamente enumerables

1	Sensible al Contexto	$\alpha A \beta \rightarrow \alpha \gamma \beta, \gamma \geq 1$	Autómata Linealmente Acotado	$\{ a^n b^n c^n \mid n \geq 1 \}$, concordancia gramatical
2	Libre de Contexto (GLC)	$A \rightarrow \alpha$ (A único no-terminal)	Autómata de Pila (PDA)	Paréntesis balanceados, sintaxis Python/Java
3	Regular	$A \rightarrow aB$ o $A \rightarrow a$	Autómata Finito (AFD/AFND)	Identificadores, tokens léxicos, emails

Aplicación en Compiladores

La Jerarquía de Chomsky no es únicamente un constructo teórico; su aplicación más directa y relevante se encuentra en la arquitectura de los compiladores modernos. Cada fase del proceso de compilación corresponde a un nivel específico de la jerarquía, y esta correspondencia no es accidental: determina qué tipo de autómata y qué algoritmos deben emplearse en cada etapa.

Análisis Léxico — Tipo 3

La primera fase de la compilación es el análisis léxico. El lexer (o scanner) toma el código fuente como una secuencia de caracteres y la convierte en una secuencia de tokens. Por ejemplo, la instrucción `if (x > 0)` produce los tokens: [IF], [LPAREN], [ID:x], [GT], [NUM:0], [RPAREN]. Este proceso se implementa con gramáticas Tipo 3 y sus correspondientes autómatas finitos. Herramientas como Flex y Lex generan automáticamente el código del analizador léxico a partir de especificaciones en expresiones regulares.

Análisis Sintáctico — Tipo 2

La segunda fase es el análisis sintáctico. El parser toma la secuencia de tokens y verifica que su estructura siga la gramática del lenguaje de programación, construyendo el

Árbol de Sintaxis Abstracta (AST). Este árbol representa la estructura jerárquica del programa. Las gramáticas Tipo 2 (GLC) son suficientes y necesarias para esta tarea, dado que los lenguajes de programación tienen estructuras anidadas (bloques, expresiones, funciones) que los autómatas finitos no pueden manejar. Herramientas como ANTLR, Yacc y Bison automatizan la generación de parsers a partir de GLC.

Análisis Semántico — Tipos 0 y 1 en teoría

El análisis semántico verifica condiciones que escapan al alcance de las GLC, como que una variable está declarada antes de ser usada, o que los tipos de datos en una operación sean compatibles. Formalmente, estas verificaciones corresponden a gramáticas Tipo 1 o Tipo 0. Sin embargo, en la práctica, los compiladores implementan estas verificaciones mediante algoritmos propios (tablas de símbolos, verificación de tipos) en lugar de usar directamente gramáticas sensibles al contexto, dado que su procesamiento sería computacionalmente costoso.

Simulador Visual Interactivo

Descripción del Programa

Como complemento práctico a los fundamentos teóricos, se desarrolló un simulador visual interactivo en Python utilizando la biblioteca Tkinter. El programa denominado 'Jerarquía de Chomsky — Simulador Visual' permite al usuario interactuar con los cuatro tipos de gramáticas de manera directa, ingresando cadenas de texto y verificando su pertenencia a cada tipo de lenguaje formal.

Funcionalidades

- Tipo 0: Acepta cualquier cadena, ilustrando el poder irrestricto de la Máquina de Turing.

- Tipo 1: Valida cadenas de la forma $a^n b^n c^n$ (ejemplo clásico de lenguaje sensible al contexto).
- Tipo 2: Verifica el balanceo de paréntesis, demostración del poder del Autómata de Pila.
- Tipo 3: Valida identificadores según la regla: inicio con letra o '_', seguido de letras, dígitos o '_'.
- Generador de ejemplos: Produce cadenas válidas automáticamente para cada tipo.

Código Fuente

El código fuente completo del simulador, implementado en Python 3 con Tkinter, se presenta a continuación. El programa está estructurado como una clase orientada a objetos (ChomskyVisualApp) que instancia un notebook con cuatro pestañas, una por cada tipo de gramática:


```

# Jerarquia_de_chomsky.py - Simulador Visual
# Lenguaje y Compiladores 2026-I | UNEG | Arnaldo Perdomo

import tkinter as tk
from tkinter import ttk, messagebox
import random

class ChomskyVisualApp:
    def __init__(self, root):
        self.root = root
        self.root.title('Jerarquia de Chomsky - Simulador Visual')
        self.root.geometry('1420x880')
        self.root.configure(bg='#0a1428')
        # ... (ver código fuente completo adjunto en repositorio)

    def validate(self, tipo, entry, result):
        cadena = entry.get().strip()
        # Tipo 3: Identificadores
        if tipo == 3: ok = self.is_valid_id(cadena)
        # Tipo 2: Paréntesis balanceados
        elif tipo == 2: ok = self.balanced_parentheses(cadena)
        # Tipo 1:  $a^nb^nc^n$ 
        elif tipo == 1: ok = self.is_an_bncn(cadena)
        # Tipo 0: Siempre acepta
        else: result.insert('end', 'Cualquier cadena es potencialmente aceptable.')

    def is_valid_id(self, s):
        if not s: return False
        return (s[0].isalpha() or s[0] == '_') and all(c.isalnum() or c == '_' for c in s)

    def balanced_parentheses(self, s):
        count = 0
        for c in s:
            if c == '(': count += 1
            elif c == ')': count -= 1
            if count < 0: return False

```

```

return count == 0

def is_an_bncn(self, s):
    # Verifica cadenas de la forma a^n b^n c^n
    try:
        a, rest = s.split('b', 1)
        b, c = rest.split('c', 1)
        return len(a) == len(b) == len(c) > 0
    except: return False

```

Capturas de Pantalla del Simulador

A continuación, se presentan capturas del simulador en ejecución para cada tipo de gramática:



Jerarquía de Chomsky

Simulador Visual Interactivo • Tema 3 - Lenguaje y Compiladores | UNEG

Tipo 0 — Sin Restricción Tipo 1 — Sensible al Contexto Tipo 2 — Libre de Contexto Tipo 3 — Regular

TIPO 1 — Sensible al Contexto

Explicación y Poder Computacional

Requiere memoria proporcional a la entrada
 Autómata: Autómata Linealmente Acotado (ALA)

Características:

- Producciones: $\alpha\beta \rightarrow \gamma\delta$ ($|\gamma| \geq 1$)
- No puede reducir la longitud de la cadena

Ejemplo clásico: $\{ a^n b^n c^n \mid n \geq 1 \}$
 Concordancia gramatical (el gato / los gatos).

Prueba el Lenguaje

Ingresar una cadena:

Validar Generar Ejemplo

Jerarquía de Chomsky

Simulador Visual Interactivo • Tema 3 - Lenguaje y Compiladores | UNEG

Tipo 0 — Sin Restricción Tipo 1 — Sensible al Contexto Tipo 2 — Libre de Contexto Tipo 3 — Regular

TIPO 1 — Sensible al Contexto

Explicación y Poder Computacional

Requiere memoria proporcional a la entrada
 Autómata: Autómata Linealmente Acotado (ALA)

Características:

- Producciones: $\alpha\beta \rightarrow \gamma\delta$ ($|\gamma| \geq 1$)
- No puede reducir la longitud de la cadena

Ejemplo clásico: $\{ a^n b^n c^n \mid n \geq 1 \}$
 Concordancia gramatical (el gato / los gatos).

Prueba el Lenguaje

Ingresar una cadena:

Validar Generar Ejemplo

Cadena: aaabbbcccc

☒ TIPO 1 (CSL): Válida ($a^n b^n c^n$)
 Ejemplo clásico de gramática sensible al contexto

Jerarquía de Chomsky

Simulador Visual Interactivo • Tema 3 - Lenguaje y Compiladores | UNEG

Tipo 0 — Sin Restricción Tipo 1 — Sensible al Contexto Tipo 2 — Libre de Contexto Tipo 3 — Regular

TIPO 2 — Libre de Contexto

Explicación y Poder Computacional

Memoria en pila (stack)
 Autómata: Autómata de Pila (PDA)

Características:

- Producciones: $A \rightarrow \alpha$ (A único no-terminal)
- Ideal para estructuras jerárquicas y recursivas

Ejemplos comunes:

- Paréntesis balanceados
- Expresiones aritméticas
- Sintaxis de lenguajes de programación.

Prueba el Lenguaje

Ingresar una cadena:

Validar Generar Ejemplo

Jerarquía de Chomsky

Simulador Visual Interactivo • Tema 3 - Lenguaje y Compiladores | UNEG

Tipo 0 — Sin Restricción Tipo 1 — Sensible al Contexto Tipo 2 — Libre de Contexto Tipo 3 — Regular

TIPO 2 — Libre de Contexto

Explicación y Poder Computacional

Memoria en pila (stack)
 Autómata: Autómata de Pila (PDA)

Características:

- Producciones: $A \rightarrow \alpha$ (A único no-terminal)
- Ideal para estructuras jerárquicas y recursivas

Ejemplos comunes:

- Paréntesis balanceados
- Expresiones aritméticas
- Sintaxis de lenguajes de programación.

Prueba el Lenguaje

Ingresar una cadena:

Validar Generar Ejemplo

Cadena: ((a + b) * (c - d))

☒ TIPO 2 (GLC): Paréntesis balanceados
 Común en sintaxis de lenguajes de programación

2.2.1 Gramáticas y Geometría Computacional

La relación entre el lenguaje y la forma ha sido un pilar fundamental en la computación gráfica. La capacidad de describir un objeto complejo mediante una cadena de caracteres finita es lo que permite la compresión y generación de estructuras fractales.

Un Lenguaje de Dominio Específico (DSL) es un lenguaje de programación especializado en un dominio de aplicación particular, a diferencia de los lenguajes de propósito general (GPL) como C++ o Python.

- Naturaleza: Los DSL ofrecen una mayor expresividad y facilidad de uso dentro de su ámbito, ya que utilizan abstracciones y notaciones que corresponden directamente a los conceptos del dominio.
- En el contexto de la tortuga: El lenguaje de comandos de la tortuga es un DSL diseñado para el dominio de la geometría y la lógica algorítmica. Sus primitivas (avanzar, girar, repetir) encapsulan operaciones matemáticas complejas en comandos intuitivos.

Para el control del motor gráfico, se ha definido un alfabeto: $\Sigma = \{a, c, g, t\}$, este conjunto de terminales es suficiente para describir la mayoría de los polígonos regulares y patrones lineales. La gramática aquí se define por la secuencia lineal de estos símbolos.

- Variables de movimiento: $\{a, g, t, v, y, z\}$ que transforman el estado actual $(x, y, \angle)$
- Variables de marcado: que permite insertar nodos o puntos críticos sin alterar la posición del cursor. $\{c\}$

Geometría de la Tortuga (Turtle Geometry)

El sistema opera bajo una lógica egocéntrica. El estado de la tortuga se define mediante su posición y su orientación angular

- Independencia del sistema de coordenadas: El usuario no necesita definir un plano cartesiano. Al enviar la secuencia avavav, el sistema interpreta que cada a implica un desplazamiento relativo a la dirección actual y cada v una actualización de esa dirección.
- Transformación de estado: Cada comando ejecuta una función matemática

Análisis Semántico y Sintáctico

El programa actúa como un intérprete que procesa la cadena de entrada mediante un bucle de análisis léxico.

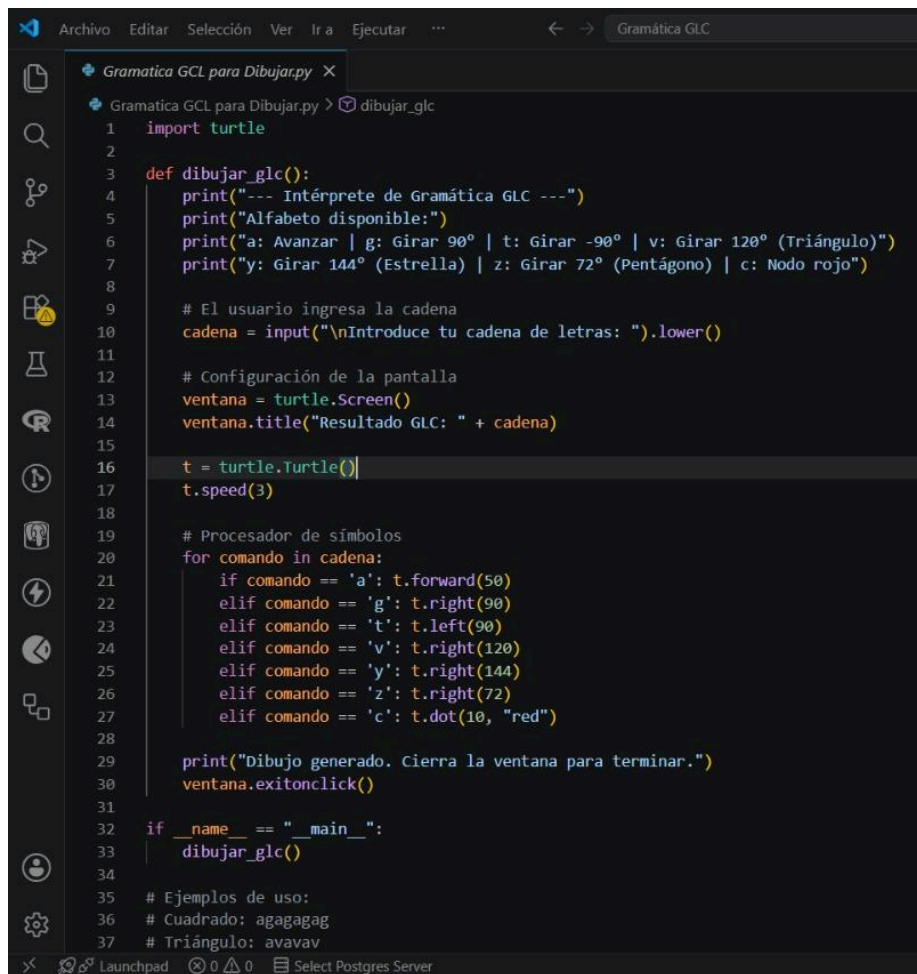
1. Sintaxis: Se valida que cada carácter pertenezca al alfabeto Σ
2. Semántica: Cada símbolo posee un significado geométrico único. Por ejemplo, el símbolo y (144°) es semánticamente equivalente a la rotación necesaria para generar una punta de estrella, mientras que z (72°) lo es para un pentágono.

Fundamentos Matemáticos: El Cierre de Polígonos

La capacidad de generar figuras cerradas (polígonos regulares) se basa en la suma de giros externos.

Este sistema demuestra que la **sintaxis** puede controlar completamente la **semántica visual**. Al eliminar operadores de memoria (como los corchetes), el sistema simplifica la arquitectura del intérprete a un modelo de ejecución secuencial, lo cual es ideal para el dibujo de formas

cerradas, figuras geométricas básicas y rutas de nodos, manteniendo una alta legibilidad para el usuario y una lógica de ejecución predecible y eficiente.



```

1  import turtle
2
3  def dibujar_glc():
4      print("--- Intérprete de Gramática GLC ---")
5      print("Alfabeto disponible:")
6      print("a: Avanzar | g: Girar 90° | t: Girar -90° | v: Girar 120° (Triángulo)")
7      print("y: Girar 144° (Estrella) | z: Girar 72° (Pentágono) | c: Nodo rojo")
8
9      # El usuario ingresa la cadena
10     cadena = input("\nIntroduce tu cadena de letras: ").lower()
11
12     # Configuración de la pantalla
13     ventana = turtle.Screen()
14     ventana.title("Resultado GLC: " + cadena)
15
16     t = turtle.Turtle()
17     t.speed(3)
18
19     # Procesador de símbolos
20     for comando in cadena:
21         if comando == 'a': t.forward(50)
22         elif comando == 'g': t.right(90)
23         elif comando == 't': t.left(90)
24         elif comando == 'v': t.right(120)
25         elif comando == 'y': t.right(144)
26         elif comando == 'z': t.right(72)
27         elif comando == 'c': t.dot(10, "red")
28
29     print("Dibujo generado. Cierra la ventana para terminar.")
30     ventana.exitonclick()
31
32 if __name__ == "__main__":
33     dibujar_glc()
34
35 # Ejemplos de uso:
36 # Cuadrado: agagagag
37 # Triángulo: avavav

```

2.3.- Higiene y Optimización de Gramáticas

2.3.1.- Patologías de las Gramáticas

En el diseño de compiladores e intérpretes, las gramáticas mal estructuradas introducen deficiencias críticas denominadas patologías gramaticales. Estas anomalías impiden un análisis sintáctico determinístico y eficiente, arrastrando al compilador a bucles infinitos, indeterminismo o a la generación de árboles de sintaxis contradictorios ante un mismo código fuente.

A continuación, se analizan y resuelven los tres casos prácticos fundamentales:

a) Una Gramática Ambigua

Una gramática es ambigua si existe al menos una cadena válida dentro de su lenguaje que admite dos o más árboles de derivación independientes (o dos derivaciones por la izquierda/derecha distintas). Para un compilador, esto es inaceptable, ya que una misma línea de código podría interpretarse de dos formas semánticas diferentes.

Caso Práctico

Sea una gramática aritmética simplificada G_1 para procesar expresiones de suma y multiplicación, definida por las siguientes producciones:

$$E \rightarrow E + E \mid E \times E \mid id$$

Donde E es el símbolo no terminal (expresión) e id representa un terminal (un número o variable).

Demostración de Ambigüedad

Se evaluará la cadena legítima: $id + id \times id$

Al aplicar las reglas de producción, se comprueba que es posible ramificar la estructura sintáctica de dos maneras completamente distintas, alterando la precedencia de los operadores:

Árbol de Derivación 1 (Prioriza la multiplicación)

Este árbol interpreta que la suma se ejecuta al final: $(id + (id \times id))$

$$\begin{array}{c}
 E \\
 / \quad | \quad \backslash \\
 E + E \\
 | \quad / \quad | \quad \backslash \\
 id \quad E \times E
 \end{array}$$

$$\begin{array}{cc} | & | \\ \text{id} & \text{id} \end{array}$$

Árbol de Derivación 2 (Prioriza la suma)

Este árbol interpreta equívocamente que la suma tiene mayor prioridad que la multiplicación: $((\text{id} + \text{id}) \times \text{id})$

$$\begin{array}{ccccc} & & \mathbf{E} & & \\ & / & | & \backslash & \\ & \mathbf{E} & \times & \mathbf{E} & \\ & / & | & \backslash & | \\ & \mathbf{E} & + & \mathbf{E} & \text{id} \\ & | & & | & \\ & \text{id} & & \text{id} & \end{array}$$

Conclusión del caso: Al demostrarse la existencia de dos árboles estructurales diferentes para una misma cadena, se certifica formalmente que la gramática G_1 es ambigua.

b) Recursividad por la Izquierda

Una regla de producción posee recursividad por la izquierda si el símbolo no terminal del lado izquierdo de la producción aparece de forma inmediata como el primer símbolo del lado derecho.

Matemáticamente se expresa como: $\mathbf{A} \rightarrow \mathbf{A}\alpha$. Esto causa una patología crítica en los analizadores sintácticos descendentes (como los parsers *Top-Down* LL(1)), induciendo al algoritmo a bucles infinitos de sustitución de los cuales nunca puede salir para consumir un símbolo terminal.

Caso Práctico

Sea la gramática G_2 con recursividad por la izquierda:

$$\mathbf{E} \rightarrow \mathbf{E} + \mathbf{T} \mid \mathbf{T}$$

Aquí, $\alpha = + T$ y $\beta = T$ (la alternativa que no empieza con E).

Algoritmo Paso a Paso para Eliminar la Recursividad por la Izquierda

Para solucionar esta patología, se aplica la ley de transformación matemática para reglas del tipo $A \rightarrow A\alpha \mid \beta$. El algoritmo sustituye la recursividad por la izquierda por una **recursividad por la derecha** equivalente, introduciendo una nueva variable intermedia (A'):

$$\begin{aligned} A &\rightarrow \beta A' \\ A' &\rightarrow \alpha A' \mid \epsilon \end{aligned}$$

(Donde ϵ representa la cadena vacía).

- **Paso 1: Identificar las variables del patrón.** En G_2 , el no terminal conflictivo es $A = E$. El sufijo recursivo es $\alpha = + T$, y la base limpia es $\beta = T$.
- **Paso 2: Crear el nuevo símbolo no terminal.** Se genera la variable auxiliar E' .
- **Paso 3: Reestructurar la producción principal (E).** Se obliga a la gramática a derivar primero la estructura terminal o base (β) seguida del nuevo estado:

$$E \rightarrow T E'$$

- **Paso 4: Definir las producciones de la variable auxiliar (E').** Se traslada la operación recursiva al final del lado derecho, añadiendo la opción de parada (ϵ):

$$E' \rightarrow + T E' \mid \epsilon$$

Gramática Resultante Optimizada

La gramática limpia y procesable de forma descendente queda definida como:

$$E \rightarrow T E'$$

$$E' \rightarrow + T E' \mid \epsilon$$

c) Factorización por la Izquierda

La necesidad de factorización por la izquierda surge cuando dos o más reglas de producción que parten de un mismo símbolo no terminal comparten el mismo prefijo inicial en sus lados derechos.

Esta patología genera indeterminismo; cuando el analizador sintáctico lee un token del código fuente, no posee la información suficiente para predecir cuál de las dos reglas debe expandir, viéndose obligado a realizar procesos ineficientes de *backtracking* (vuelta atrás).

Caso Práctico (Estructura de Control)

Considérese una gramática común para sentencias condicionales donde el bloque **else** es opcional:

$$S \rightarrow \text{if } C \text{ then } S \text{ else } S \mid \text{if } C \text{ then } S \mid \text{enunciado}$$

Donde S es sentencia y C es condición. Ambas producciones condicionales comparten exactamente el mismo prefijo inicial: $\alpha = \text{if } C \text{ then } S$.

Algoritmo de Optimización

El algoritmo consiste en extraer el factor común (prefijo α) y postergar la decisión sintáctica introduciendo un nuevo símbolo no terminal (S') que maneje exclusivamente los sufijos remanentes.

La estructura matemática dicta que si $A \rightarrow \alpha\beta_1 \mid \alpha\beta_2$, se transforma en:

$$A \rightarrow \alpha A'$$

$$A' \rightarrow \beta_1 \mid \beta_2$$

- **Paso 1: Aislar el factor común.** El prefijo idéntico es $\alpha = \text{if } C \text{ then } S$.

- **Paso 2: Identificar los remanentes (β_n).**

Para la primera producción, el remanente es $\beta_1 = \text{else } S$. Para la segunda producción, el remanente es la cadena vacía, por lo que $\beta_2 = \epsilon$.

- **Paso 3: Construir la gramática factorizada.** Se reescribe la producción original apuntando al factor común y a una nueva variable S' :

Gramática Resultante Optimizada

$$S \rightarrow \text{if } C \text{ then } S \ S' \mid \text{enunciado}$$

$$S' \rightarrow \text{else } S \mid \epsilon$$

Con esta estructura optimizada, el compilador puede consumir con total certeza el prefijo **if C then S**, y solo cuando llegue al siguiente token decidirá, de forma determinística, si se expande hacia un bloque else o si finaliza la instrucción mediante la cadena vacía (ϵ).

2.4 PGN Ajedrez + Regex + AFD

2.4.1. Notación PGN (Portable Game Notation)

La Notación de Partidas de Ajedrez (PGN) representa un estándar esencial para el registro de partidas. Dada la necesidad de procesar estas cadenas con precisión, se aplican técnicas de lenguajes formales para modelar su reconocimiento. En esta sección, se plantea y ejecuta un modelo basado en autómatas finitos capaz de validar la sintaxis de un subconjunto representativo de movimientos de ajedrez

2.4.2. Definición de un subconjunto simplificado del lenguaje PGN

El Portable Game Notation (PGN) es un formato estándar para el registro de partidas de ajedrez. Dado que su especificación completa es extensa, para esta investigación definiremos un subconjunto formal (L_{PGN}) que contempla los movimientos básicos de piezas y peones, permitiendo validar la estructura sintáctica de una jugada aislada.

Este subconjunto se basa en el alfabeto $\Sigma = \{a, b, c, d, e, f, g, h, 1, 2, 3, 4, 5, 6, 7, 8, N, B, R, Q, K\}$, donde:

- **Columnas:** $\{a, \dots, h\}$
- **Filas:** $\{1, \dots, 8\}$
- **Piezas:** N(Caballo), B (Alfil), R (Torre), Q (Reina), K (Rey). Los movimientos de peón se identifican por la ausencia de letra de pieza.

Especificación de Cadenas Válidas e Inválidas:

Para validar el autómata, consideramos las siguientes instancias:

Categoría	Ejemplo	Descripción
Válido	e4	Movimiento de peón a la columna 'e', fila 4.
Válido	Nf3	Movimiento de Caballo a la columna 'f', fila 3.
Inválido	z9	Columna o fila fuera de los límites del tablero.
Inválido	N1	Falta la columna de destino en el movimiento de la pieza.
Inválido	e9	Fila inexistente en el tablero estándar.

2.4.3. Diseño de la Expresión Regular (Regex)

Para el subconjunto definido anteriormente, el diseño se fundamenta en la capacidad de las expresiones regulares para identificar tokens específicos.

A. Expresión Regular (Regex)

La expresión regular que define nuestro subconjunto de movimientos básicos es:

`([NBRQK][a-h][1-8]|[a-h][1-8])`

- **Explicación técnica:**

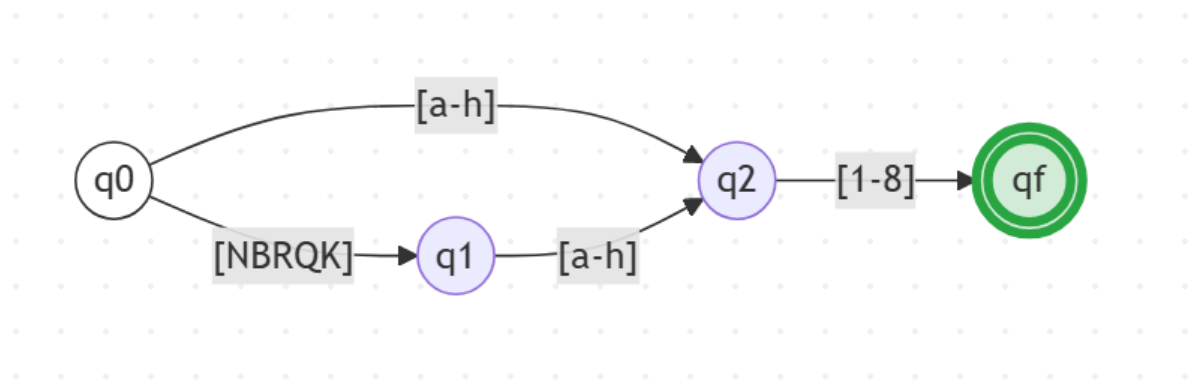
- **[NBRQK]:** Reconoce el carácter inicial de la pieza (Caballo, Alfil, Torre, Reina o Rey).
- **[a-h]:** Reconoce la columna de destino.
- **[1-8]:** Reconoce la fila de destino.
- **|:** Operador lógico "OR" que permite distinguir entre un movimiento de pieza con nombre o un movimiento simple de peón (columna + fila).

B. Autómata Finito Determinístico (AFD)

- **Nota para tu informe:** Aquí debes incluir el gráfico.
- **Diseño lógico:**
 - **Estado inicial (q_0):** Espera el primer carácter.
 - **Transiciones:** Si recibe una letra de pieza, pasa a un estado intermedio que espera columna y fila. Si recibe directamente una columna (peón), pasa a esperar la fila.
 - **Estado de aceptación (q_f):** Se alcanza sólo cuando la cadena cumple con el formato exacto de [Pieza]+[Columna]+[Fila] o [Columna]+[Fila].

Diagrama del AFD (Representación Lógica)

El autómata se define por el conjunto de estados $Q = \{q_0, q_1, q_2, q_3, q_f\}$ y las transiciones que procesan las cadenas [Pieza][Columna][Fila] o [Columna][Fila].



Estado q_0 (Inicial): El autómata espera el primer carácter de la cadena.

Estado q_1 (Pieza detectada): Se alcanza si el primer carácter es $\{N, B, R, Q, K\}$. Indica que estamos procesando un movimiento de pieza mayor.

Estado q_2 (Columna detectada): Se alcanza si el primer carácter fue una columna $\{a-h\}$ (movimiento de peón) o si venimos de q_1 tras recibir una columna.

Estado q_f (Aceptación): Se alcanza solo tras recibir una fila $\{1-8\}$ válida. Indica que la cadena ha sido reconocida como una jugada sintácticamente correcta según nuestro subconjunto.

```

import re

def procesar_jugada(jugada: str) -> bool:
    """
    Valida una jugada de ajedrez basada en el subconjunto PGN definido.

    Lógica de estados:
    - q0: Estado inicial.
    - q1: Se recibió una pieza (N, B, R, Q, K).
    - q2: Se recibió una columna (a-h).
    - qf: Estado de aceptación tras recibir fila (1-8).
    """
  
```

```
# Expresión regular que representa las transiciones del AFD
# [Pieza][Columna][Fila] | [Columna][Fila]
patron = r'^([NBRQK][a-h][1-8]|[a-h][1-8])$'

return bool(re.match(patron, jugada))

# Pruebas del AFD
if __name__ == "__main__":
    pruebas = ["e4", "Nf3", "Ra8", "z9", "N1", "e9"]
    for p in pruebas:
        resultado = "Aceptada" if procesar_jugada(p) else "Rechazada"
        print(f"Jugada: {p} -> {resultado}")
```

Conclusión

El estudio y la implementación práctica de los lenguajes y gramáticas formales revelan su papel central en la informática teórica y aplicada. A lo largo de este informe, se ha confirmado que la jerarquía de Chomsky no es una mera clasificación académica, sino un instrumento operativo que guía el diseño de compiladores y otras herramientas de procesamiento de lenguajes. La correspondencia entre cada tipo gramatical y un autómata específico determina los algoritmos y estructuras de datos más adecuados para cada fase de la compilación, desde el análisis léxico, basado en autómatas finitos y expresiones regulares, hasta el análisis sintáctico, que emplea autómatas de pila y gramáticas libres de contexto para construir árboles de sintaxis abstracta. A su vez, las verificaciones semánticas, aunque teóricamente asociadas a gramáticas sensibles al contexto, se resuelven en la práctica mediante tablas de símbolos y sistemas de tipos, lo que demuestra la flexibilidad de los enfoques híbridos en la ingeniería de compiladores.

La extensión de estos conceptos a dominios específicos, como la geometría computacional mediante lenguajes de dominio específico (DSL) para el control de tortugas gráficas, ilustra la versatilidad de los formalismos gramaticales: una secuencia sencilla de comandos puede generar figuras complejas, y la semántica visual queda completamente determinada por la sintaxis de la entrada. De manera análoga, la aplicación de expresiones regulares y autómatas finitos a la notación PGN de ajedrez demuestra que incluso subconjuntos simplificados de lenguajes estándar pueden ser validados eficientemente con modelos de memoria constante, lo que resulta útil en sistemas de anotación y análisis de partidas.

En cuanto a la higiene y optimización de gramáticas, se ha abordado el problema de la ambigüedad, la recursividad por la izquierda y la factorización por la izquierda, proporcionando algoritmos claros para transformar gramáticas problemáticas en equivalentes

no ambiguas y determinísticas. Estas técnicas son indispensables para la construcción de analizadores sintácticos descendentes que eviten retrocesos y bucles infinitos, garantizando así un procesamiento eficiente y fiable del código fuente.

Finalmente, el simulador visual desarrollado en Python constituye un recurso didáctico valioso que permite a los estudiantes y profesionales experimentar de manera interactiva con los distintos niveles de la jerarquía, validando cadenas y comprendiendo las capacidades y limitaciones de cada tipo gramatical. Esta herramienta refuerza el aprendizaje al hacer tangible la teoría y pone de manifiesto la relevancia de los lenguajes formales en la formación de ingenieros informáticos. En conjunto, el trabajo realizado no solo consolida los conocimientos fundamentales, sino que también abre la puerta a aplicaciones más avanzadas, como el diseño de nuevos lenguajes de dominio específico y la optimización de procesadores de lenguajes, reafirmando que la teoría de lenguajes formales sigue siendo un campo vivo y de gran proyección práctica en la era del software.

Referencias

Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2007). *Compilers: Principles, techniques, and tools* (2nd ed.). Pearson Addison-Wesley.

Chomsky, N. (1956). Three models for the description of language. *IRE Transactions on Information Theory*, 2(3), 113-124. <https://doi.org/10.1109/TIT.1956.1056813>

Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2007). *Introduction to automata theory, languages, and computation* (3rd ed.). Pearson Addison-Wesley.

Sipser, M. (2013). *Introduction to the theory of computation* (3rd ed.). Cengage Learning

Python Software Foundation. (2026). *Python Language Reference* (v3.12).
<https://docs.python.org/3/>

Standardization, I. O. F. (1996). *ISO/IEC 14977: Information technology — Syntactic metalanguage — Extended BNF*. International Organization for Standardization.

Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2007). *Introduction to Automata Theory, Languages, and Computation*. Pearson.

Sipser, M. (2012). *Introduction to the Theory of Computation*. Cengage Learning.